ADR-002c. Northlake demo service bundle

Source bundle implementation checkpoint and research inventory for remaining demo services

DATE	November 1, 2026
PREPARED BY	Energy Hippo implementation team
AUDIENCE	Northlake implementation and EnergyHippo engineering
STATUS	Source bundle implemented and locally verified; installed Northlake acceptance and remaining service packages pending

Purpose and proposed direction

Build a portable collection of synthetic source services around the existing Northlake database and installed EEMSuite applications. The bundle must generate the files and responses that the real collectors, importers, document paths and workflows consume. It must also host believable provider pages and a Northlake site that demonstrates API and Power BI integration. Deliver this incrementally alongside Northlake's acceptance runs, rather than requiring every simulator before the first useful run.

The proposed unit of deployment is **one versioned Compose bundle**, with one main Northlake source image and protocol-specific services where needed. It is not one container per gateway, and it does not require putting the EEMSuite applications or SQL Server into containers. Reuse the identity stack and Mailpit. Reuse the existing PDF generators. Keep ENERGY STAR on its existing external testing path; do not build an ENERGY STAR emulator. EnergyAI is optional and low priority.

This record extends the subject of EnergyHippo's ADR-002, ADR-002a and ADR-002b from authentication tooling to the broader demo environment. It does not replace their identity decisions. The full research lives with Northlake's Stage 11 external-services work; Stage 3 is its first delivery consumer. Read the Stage 3 recipe, bill stage, operations stage and third-party research alongside it.

The initial research below was followed by the user-authorized source-bundle implementation recorded here. Its original inventory remains a dated snapshot, not a statement of the current Docker or product-source state. `CustomDevelopment` was excluded and unrelated checkout changes were preserved throughout.

Implemented source bundle checkpoint

The runnable package is services/sources. It has its own Compose project, a Python producer image, a Caddy image and a persistent named volume. It serves provider pages at `http://localhost:18081/` locally. Source manifests, generators, fonts and shared Northlake branding are baked into the image; generated history is written to the volume. The destination server needs the two images and Compose configuration, with no repository checkout or external data directory required for HTTP use.

Delivered capability	Verified scope
File gateways	215 direct inputs across seven profiles and 14 nodes: AcquiSuite, MDEF, MV9, Spinwave, generic FIG, FIXEDNETWORKS and Ridgeline SCADA. All seven FIG encodings have compatibility fixtures.
Historical model	December 2023 supporting data, January 2024 onward and source lifetime boundaries, including Ridgeline beginning July 2024. Seed 104729; stable source/channel/time keys; native intervals and Pacific DST.
Billing	Complete issuable series for 50 supplier/internal-allocation accounts, PDFs, canonical manifests, normalized upload JSON, BIF and native XML; VED/RCU ordered provider CSV and matching schemas. Ridgeline's own statement remains a Stage 7 reference.
Tariffs	Explicit 2024-2026 supplier tariff documents in the initial release; future annual versions are generated as needed. Existing 2024/2025 terms are preserved. General monetary escalation is 3% annually from 2026, retaining Helios's 2% PPA and fixed export credit and Ridgeline's fixed agreement.
Publication	Daily bootstrap partitions and subsequent fifteen-minute UTC windows; immutable artifact URLs, hashes, metadata and atomic publication. Persisted cutoff, source identity and watermarks survive restart. Hourly records wait for the hour to close.
Delivery	One CLI supports shared-directory copy and HTTP synchronization. Atomic writes and durable receipts prevent an EEM-consumed file from being sent again. Explicit replay is required; interrupted uncertain transfers require reconciliation. Continuous delivery is opt-in.
Provider pages	Read-only provider, account, year and month navigation, paginated file history and downloads from the same immutable catalog. The Thermal Plant is an internal catalog, with no invented public identity.

All 24 existing sample PDFs and the AcquiSuite complete/gap/backfill fixtures remain unchanged. Named estimates, corrections, missing invoices, renumbering, outages and transfers remain explicit source cases. Ordinary delivery selects accepted revisions; earlier/corrected acceptance cases require deliberate selection. Open billing periods are not issued early, and December lead-in data does not create December-posted invoices.

Decisions made while implementing

- The user approved synthetic winter supplier part-peak **08:00-22:00 on weekdays**. The proposed 17:00-21:00 window could not contain the Student Center anchor's energy below its stated demand maximum. Summer windows and holiday treatment remain explicit; Stage 4's simplified analysis schedules were not changed.
- Versioned assumptions specify a 2,200 kW nominal Central Plant process load, honoring its existing description as the largest estate service. Cedar apartment and house shares conserve the master readings. July wind generation, possible energy, exports and station service honor the original source totals and outage.
- Known water register readings bound prior usage with a documented positive opening balance. Source readings, multipliers, replacements and invoice quantities share one measurement timeline. Unmetered charges and logical transfers do not invent physical consumption.
- Newly generated invoice identities use `_generated_YYYY_MM`. This avoids collisions with original anchor names that sometimes use service-start months rather than posting months. Existing invoice identities and absent extracted invoice numbers remain intact.
- New PDF rows wrap long readings and meter labels without reducing precision. The existing renderer's default behavior and all anchored PDF bytes remain unchanged.

Narrow EnergyHippo importer changes

Provider CSV conversion now preserves signed credits and decides zero-line inclusion independently for each item. Native XML numeric fields use decimal precision, preventing large-credit cents and precise quantities from being rounded through floats. A narrow deserialization normalization preserves existing scientific-notation payload compatibility for both native XML and the BIF-to-XML path. XML element names and the positional BIF format are unchanged.

Native XML carries supported per-meter dates, readings and multipliers only where the item association is unambiguous. CSV/BIF do not claim to preserve fields absent from their legacy tuple. Estimate status remains in PDFs and source evidence; structured product persistence is still a capability gap. Decimal XML parsing also does not establish decimal storage in the installed BillItem reading columns. Deployment-specific upload aliases and stored image paths remain integration concerns.

Source-side verification

The full bootstrap completed through **2026-11-02 06:15 UTC (November 1, 22:15 Pacific)**. The release is `northlake-sources-v1-5a4452b39320-104729`.

Check	Result
Full generation sizing	25,955 artifacts; 591,560,045 artifact bytes; 804.3 seconds; peak RSS 203,396 KiB (about 199 MiB), within a 768 MiB container limit.
Expanded history	13,508,776 gateway readings; 1,673 supplier/internal invoices across 50 accounts; 28 separate Ridgeline statement references; 18 annual tariff documents plus JSON.
Catalog integrity	All 25,955 published hashes verified, with no missing or changed artifacts.
Whole-history financial audit	7,798 financial artifacts downloaded and hash-checked. All 1,673 native XML, BIF and normalized JSON representations, plus 1,036 provider CSV files, reconciled with their manifests. All 147 annual tariff values independently recalculated; 26 issued transfer pairs matched, with the latest credit retained in transit until its receiving cycle.
Python behavioral checks	64 passing tests cover model constraints, tariffs, source lifetimes, registers, revisions, DST/leap day, incremental equivalence, publication, delivery receipts and page navigation.
Real gateway collectors	Opt-in headless bridge passed all included formats, the seven FIG variants and original AcquiSuite fixtures. The largest bridge run parsed 93 files and 528,046 readings, including July wind, outage and DST/repeated-hour cases.
Bill importer	31 focused headless tests passed with zero skips. Actual CSV conversion, BIF writing/parsing, native XML precision, signed credits, independent zero items and scientific notation are exercised. Scoped build had no warnings or errors.
PDF preservation	All 24 original PDF hashes preserved, including Linux image rendering comparison. Newly generated gas/electric layouts were rendered and visually inspected.
Live source service	Actual Caddy provider pages verified through the bundled in-app Browser: provider/account/year navigation, pagination, invoice and tariff download links. The running producer advanced beyond its fixed bootstrap cutoff in fifteen-minute windows.
Delivery and restart	Actual local and HTTP delivery produced catalog-identical bytes and shared receipts. Removing a consumed QA file did not trigger redelivery; explicit replay restored it. A producer restart retained all 25,955 original artifact entries and continued through 07:00 UTC; the resulting 25,995 artifacts passed hash verification.

These are source and parser checks; installed ingestion, posting, image association and application reports still need acceptance. The package runbook covers startup, delivery, verification, Windows HTTP synchronization and deployment.

Stage handoff and remaining work

Before continuous MDEF/MV9 intake, apply the explicit UTC node-setting delta in the published `contracts/source-model.json`; destination points remain Pacific. Preserve Ridgeline's fixed-PST source convention and 480-minute point offset. The source model also records source-to-gateway bindings, required consumer settings and provider-page routes. None of these installed settings were changed automatically.

Rebuild the normal EnergyHippo importer/application before accepting the importer changes. Configure the chosen source transport and destination folders deliberately, then run the Stage 3 collector checks and Stage 5 provider mapping, posting and image-association checks through the existing application paths. Keep a single active delivery transport per feed/target. EEM's FileTransfer component remains FTP-family; HTTP synchronization uses this package's helper.

The configured-empty Northlake checkpoint, schedules and stage-entry records were preserved. No historical data was imported into EEM, no bills were posted and no EEM tasks were enabled. Identity and Mailpit are unchanged. Registry publication and the final demo-server database/content restore rehearsal remain separate deployment acceptance.

Remaining packages are the 43 other direct inputs (weather, BACnet, Modbus, ODBC and Fake), handheld route protocols, document extraction, university API/Power BI integration and broader workflow/task acceptance. ENERGY STAR stays external and EnergyAI stays deferred. The following research inventory retains those broader requirements.

Original research evidence and starting point

The original research date is November 1, 2026, Pacific time. This was a read-only inspection before the implementation checkpoint above: no containers or product changes were made during that initial pass. Source roots used then were:

- **N:** `C:/Hippo/Git/hippo-synthetic-customer`, HEAD `55b9319f83671d45fc1b9d2c16b6689403fc71a6`, with substantial pre-existing working changes. Current working files, not HEAD alone, supplied the recipes and generators.
- **E:** `C:/Hippo/Git/univerus/energyhippo`, HEAD `c862a8d1cfc0f29d210e80fd6d8df9141deba8f8` at inspection. No product implementation was changed for this research.

Live SQL inspection used Windows authentication against `.` and verified server name **BVirtualPC**. Customer configuration was read from **EEMSuite_Northlake**. A separate read-only inventory of **EEMSuite** was used to identify reusable import definitions; it is not evidence that those definitions exist in Northlake. EEMSuite_MPA was not needed or inspected.

Area	Observed now	Consequence
Northlake hierarchy	51 billing accounts; 552 points	Stable source-to-account/meter/point bindings can be based on the existing hierarchy. Resolve IDs on the restored database.
Gateways	14 typed registry entries, 29 nodes and 269 bound points, including aggregate, fake and retained empty Aeris entries	Counts include more than direct acquisition. The Stage 3 recorded checkpoint identifies 258 direct bindings; this pass did not rerun that full binding checker.
Stored intervals	AnalogArchive and DigitalArchive both empty	Configuration is not ingestion acceptance. Preserve the configured-empty checkpoint before collection.
Tasks and workflows	27 workflows, 44 task definitions; only one task item, active Maintenance - Weather normalization	Built-ins exist; Northlake's gateway schedules and custom workflows still need configuration.

Area	Observed now	Consequence
Handheld	HMR_ReadType, HMR_Meter and HMR_Reading all empty	Read-type catalog and route/reading setup block Neptune/MVRS acceptance before a simulator can help.
Bills and BIP	Bill, BillItem, BIP file and staged-row tables empty; only source headers Manual Importer and Bill Importer	Those headers do not supply provider-specific parsers, mappings or imported bills.
BIP mappings and images	Utility, account/meter and commodity mappings empty; one M2 image and no bill images	Stage 5 mapping and image attachment remain work. The existing image is not bill-path proof.
My Imports	Interval Data Import, Loss Factor Import, Northlake Stage 2; seven Excel sheet setups	Stage 2 sheets exist; the MeterReadings template described in Stage 9 is not installed in Northlake.
Docker on this engine	Five identity services plus Mailpit; no other stopped containers	Identity and mail are reusable. HTTP, SFTP, RabbitMQ, protocol simulators and a document container are not deployed in this Docker context.

The identity services are Keycloak, its PostgreSQL store, Caddy, a configuration site and an identity customer test site. PostgreSQL is identity-owned existing state; the instruction to avoid an EEM SQL container does not imply removing it. Product local-service Compose files define HTTP fixtures, SFTP, RabbitMQ and Mailpit, but definitions on disk are not running services. The user clarified during this research that the document-processing container was never built. It is new work, not a service to locate or recover.

The EEMSuite comparison inventory contains Hospitality, Facility Area/Detail, Create Hierarchy, MeterSwap, Meter Reading Import and SWG/CRC/BoulderCity/NVE_Custom BIP imports. The four bill imports call `MyImportToBIP_Save`. These are references for contracts, not a request to copy customer-specific definitions or their data wholesale. Northlake needs selected definitions configured through normal supported paths.

Proposed service and artifact inventory

Capability	Reuse / build	First consumer and boundary
Identity	Reuse the five-service identity bundle and its branding	Sign-in/provisioning; preserve realm state, issuer/callback relationships and trust configuration.
Mail sink	Reuse Mailpit	Invitations, reset mail, bill notifications, alarms and scheduled reports; verify delivery and attachments.
Northlake source service	Build one image around scenario manifests and current generators	Generates gateway files, bill CSV/BIF/JSON/PDF, handheld completion files, fixture APIs and provider downloads.
Provider pages	Add pages/routes to the source service	Six fictional provider identities, tariffs, bill history, account changes and download links. A page is not evidence its importer works.
Northlake university site	Extend the existing customer-site only after reviewing its identity purpose, or add a small separate site	Public dashboard/report navigation and an authenticated server-side API client. Reuse the supplied Northlake assets.
SFTP/file delivery	Reuse product SFTP definition when transport acceptance requires it	Deliver files to the Windows worker through an explicit download/transfer workflow. An SFTP container alone does not feed a local Input directory.
Modbus simulator	New protocol service	Thermal Plant TCP registers; live polling and communication failures.

Capability	Reuse / build	First consumer and boundary
BACnet simulator	New protocol service	Thermal Plant BACnet/IP device/object values; UDP routing/discovery and polling.
Source historian	Define data contract and deployment location; use existing SQL infrastructure if selected	ODBC source queries; separate source data from the destination archive. No new SQL Server container.
Broker	Reuse RabbitMQ definition only if selected for broker acceptance	Interval publish, HMR events and queued email. Existing broker providers remain usable; RabbitMQ is not a prerequisite for every first fixture.
Document extraction	New service work; reuse current generation/extraction code where appropriate	PDF generation, extraction, normalized BIP staging and bill association are distinct steps.
Weather	Existing external NWS/ECCC/GHCN; optional Aeris coverage	Current weather/forecast plus observation history. Fully offline replay is not currently a configuration-only feature.
ENERGY STAR	Existing external testing service	No synthetic container work requested. Carry only environment configuration and acceptance prerequisites.
EnergyAI	Deferred	Consider later after enrollment/configuration gaps and service use are resolved.

One source image can share the deterministic data model and serializers across all fictional providers. Separate a service only when it needs a different protocol, runtime, network exposure or lifecycle. Do not introduce a new scheduler: EEMSuite owns task scheduling, runs, import decisions and downstream calculations. The source service owns which synthetic data is available at a selected scenario time.

Gateway coverage: actual executable paths

The current production gateway assembly contains **13 collectors**, expanding to **20 gateway/format kinds**, plus the separate HMR event publisher. FIG supplies seven formats and MV90 supplies two. Neither the 11 gateway parameter-label schemas nor the 23 values in `GatewayObjectKind` is a count of executable collectors. DataBridge is a migration alias through ODBC; no standalone GraphQL or OPC collector was found in that assembly. Aggregation is an internal computation and `OdbcFileDataGateway` exports database data to files. [E1-E7]

File acquisition

Collector / format	Source contract	Required Northlake artifact
AcquiSuite	LOG/LOG.GZ and ZIP; serial/device filename matching, UTC timestamp and configured value-column index	Extend existing complete/gap/backfill deliveries, then additional devices and history. Preserve actual serial/device/row mapping.
FIG	BEGINDATA/ENDDATA CSV; alias, timestamp and value, with supported date/time/flag variant	Cedar Row and Ridgeline series plus a small hand-checkable case.
FIG SIEMANSREPORT	Siemens CSV point declarations and date/time column map; retain exact configuration spelling	Serializer over the same canonical series.
FIG MEDIATOR	Wide CSV with timestamp header and alias columns	Valid, missing and nonnumeric-value cases.

Collector / format	Source contract	Required Northlake artifact
FIG EATON	Quoted wide CSV with site/device/meter/header composite alias	Exact composite aliases; scaling and gap cases.
FIG XML	DataRecords/Record with PointID, TimeStamp, Value and Status	Accepted Status 0 and explicit rejected-status cases.
FIG FIXEDNETWORKS	Itron MeterReadingDocument, channel IDs, interval values/status and end reads	Apartment-scale feeds; match interval alias and channel-EndRead register alias.
FIG CMEP	MEPMD01 CSV with meter/channel/UOM alias and interval triplets	Correct multiplier, base time, interval count and totals. Current parser does not use the triplet quality flags.
MV90 MDEF	Binary 216-byte record payloads, little-endian words; normal extension .mde	VED recorder/channel files, engineering values, status cases and Ridgeline period totals. This is not a SQL .mdf file.
MV90 MV9	Binary big-endian records; pulse/engineering forms and ASCII/EBCDIC handling	Helios generation feed. Stage 3's text .mv9 description is incorrect for the current parser.
Spinwave	Per-node folders; timestamp, device address and channel columns; localtime header selects local clock	Environmental readings with correct folder, address, channel and timestamp convention.

Reusable **synthetic** FIG inputs and expected JSONL already exist in E's gateway test data. `Mv90BinaryFixtureBuilder.cs` contains MDEF/MV9 builders, including endian, pulse/float, status and EBCDIC cases. Neptune/MVRS tests contain fixed-width builders. Reuse these contracts to build Northlake-owned fixture generators; do not invent the format from a filename. Two existing `.MDE` sample files identify non-Northlake customers in their parser tests and must not be bundled as synthetic customer content without provenance review. [E2-E5]

MV90 has a particularly important mapping detail: configured `RecorderID` is matched against the parsed **customer-ID field plus channel number**, not merely the channel metadata's recorder identifier. Emit the Northlake recorder alias into the field the runtime actually matches. Verify interval-end stamps, scale, units and status behavior with the resulting stored readings. [E3]

Live, query and HTTP acquisition

Collector	Source contract	Required work / limit
Modbus	TCP host/port/unit; register forms, word ordering, multiplier and additive constant; poll-time timestamps	A real protocol listener. Start with the Thermal Plant float32 register case, then disconnection and recovery. Old history needs a separate supported backfill path.
BACnet	BACnet/IP UDP, device/object types and instances, local address and optional BBMD	A real device emulator and Windows-worker reachability proof. Present-value polling does not generate two years of historical intervals.
ODBC	Named SQL/ODBC/remote-EEM source profile, query key and per-point tokens; timestamp column/mode and DaysBack	SyntheticIntervalReadings contract, correct source driver/profile, bounded query windows and replay. Source schema/location remains a decision.
Web	Legacy JSON service at BaseUrl/json with function, ContentPtID and LastTimeStamp query fields	A provider endpoint returning UTC timestamp/value arrays, NoData and error cases. This supported collector is missing from the current Stage 3 acceptance list.

Collector	Source contract	Required work / limit
NOAA / ECCC	NWS/ECCC current observations and forecasts, shared GHCN observation history, explicit measurements/units	Keep external by default. Current client restricts HTTPS hosts to api.weather.gov, www.ncei.noaa.gov and api.weather.gc.ca. A fake base URL is not configurable today.
Aeris Weather	HTTP JSON observation/archive/forecast client with configurable base URL	Optional later stub. Northlake's retained empty Aeris profile is transition state, not a required active feed.
Fake	In-product seeded synthetic values, replay and fault injection; AllowSyntheticData required	No source container. Configure its custom workflow/task and prove smoke/replay without treating it as protocol coverage.

NWS/ECCC forecast acceptance must use valid published forecast windows. Old deterministic weather is not automatically a current forecast demo. ECCC hourly forecast dew point and humidity are not supported in the present client. Preserve source attribution if caching external data. Modbus/BACnet networking and Windows-versus-Linux container hosting must be proved on the target demo host before selecting final images. [E6-E7]

Handheld routes and HMR events

Neptune and MVRS consume completed route files, rather than ordinary gateway-node point definitions. Their source of meter/channel facts is HMR configuration, keyed through meter identity and text prompt, with reading/usage points, read type, constants, decimals and rollover rules. The current empty read-type catalog and route tables are setup blockers. The requested untyped handheld gateway headers also remain an editor/setup issue in the Stage 3 recipe. Do not bypass these by fabricating direct point bindings. [N1, E4]

Build the complete cycle: export a route through the existing application; simulate completion; emit Neptune or MVRS fixed-width upload records; run the normal collector; confirm register and usage results. Include normal read, rollover, no-read/skip, correction and note/audit cases. Neptune needs an accepted ORDST timestamp. MVRS can emit an HMR event sidecar; the separate `HmrEventsPublish` component publishes JSONL events to the broker. Verify the resulting persisted HMR events as well as publication.

Replay must include the prior reading state. Some HMR modes use mutable SQL values or reading timestamps; abandoned runs for those modes are quarantined rather than freely retried. A universal replay-everything button would be misleading. [E4-E5]

Fixture production and delivery contract

Use the existing scenario/provider/bill YAML as the source of fictional facts. Add a normalized measurement and invoice model only where the current generators need one; keep serializers thin. Every artifact should carry a manifest entry with scenario/version, fixed seed, source account/meter/channel keys, UTC window, interval convention, units, file hash, delivery transport and expected downstream results. Installed integer IDs belong in a resolved binding manifest, not the source data.

Provide three deliberate modes: fixed historical acceptance cases; a bounded historical series for 2024-2025 bills and reports; and a live clock mode for ongoing demo polling. Preserve existing historic dates for the acceptance dataset. A live demo should advance a declared scenario clock rather than silently shifting invoice dates, fiscal periods and all expected totals.

The existing AcquiSuite cases are the first delivery slice: complete day **96 readings, 2,020 kWh, 120 kW peak**; gap **88 readings, 1,780 kWh**; backfill **8 readings, 240 kWh**; replay adds no new archive keys. The source uses

interval-start timestamps while EEM's relevant reports/rate paths interpret them as interval ends. The recipe explicitly records that EEM's January 15 report day therefore contains 95 of those rows and 2,010 kWh. Preserve and report that distinction; do not change fixture timestamps silently to make a report pass. [N1]

For file delivery, write to a temporary name and rename only when complete. Respect the configured age threshold (10 seconds in the current recipe). The producer owns delivery; EEM owns claim, Working, backup/archive, Error and publication handoffs. Gateway output acceptance is a chain: source claim, parse, normalized output, publication acknowledgement, stored intervals, then rendered/report totals. A successful task or HTTP response proves only its own step. [E5]

The current worker runs on Windows and reads its own filesystem. Its default gateway workspace is under instance `TaskData/<family>/<gatewayId>`; Northlake recipes override this with named `Gateways/...` paths. Bind the chosen paths explicitly rather than assuming either convention. [E5, N1]

Storage option	What it solves	Required boundary
One host delivery root	Simplest first file-feed implementation	Mount producer delivery subfolders and map task Input folders to the same Windows-visible files. Do not let the producer overwrite Working or Error.
Container-owned named volumes	No user-managed source directory	EEM must fetch files over HTTP/SFTP or receive them through a supported upload workflow; an internal volume is not a Windows input directory.
Ephemeral image data only	Rebuildable immutable seed fixtures	Insufficient for accepted uploads, source cursors, identity state and evidence that must survive recreation.

Docker-managed volumes survive container removal, but host applications need an explicit access or transfer arrangement. A bind mount is appropriate when both the host and container must access the same files. The proposed default is one optional delivery root for Stage 3, with network delivery as a later mode once its real workflow is accepted. Docker volume documentation

Bills: generation, import, parsing and images

Generate provider documents from the existing data

Northlake already has deterministic ReportLab generation in `generators/render_bills.py`, driven by all `data/bills/*.yaml` and provider tariffs. Package every bill source file, including the itemized-bills manifest, to retain the other providers' samples. Reuse it inside the source image before introducing another report engine. Extend it from sample invoices to the selected account/month series, keeping provider CSV/BIF/JSON and rendered PDF derived from the same itemized bill. The renderer must preserve billed dates, quantities, rounding, taxes, credits and invoice totals. [N2]

Telerik remains a possible later renderer if a scenario needs existing Telerik report definitions. It is not a prerequisite for initial bill PDFs. Telerik documents Linux/container support with the appropriate graphics engine, including Skia for modern .NET; fonts, native dependencies and the chosen licensed runtime still need a target-image proof. No license credentials were inspected. ReportLab already supplies the required PDF generation primitive. Telerik platform support, ReportLab PDF generation

For parsing coverage, later add clearly labeled scanned/image-only, rotated, multi-page, poor-quality and malformed documents, with an expected field manifest. Clean generated text PDFs prove a different extraction path from OCR. Do not put expected answers into the production extraction result and call that parser validation.

Custom provider sources and bill mapping

Stage 5 selects **Valley Electric District and River City Utilities** for the first provider-file imports, covering 31 accounts: 14 VED and 17 RCU. Sierra Gas, Helios and the Thermal Plant currently use manual Bill Entry; automated sources for them are later additions if selected. Ridgeline's own statement is generated by EEM in Stage 7, not imported as a supplier invoice. VED's generator and Central Plant credit-transfer lines remain on the existing VED source. [N2]

The WPF importer loads provider definitions from `Schemas/<code>.xml` plus the `BillImporter:Utilities` configuration list. It converts provider input into flat BIF; it is not a PDF parser. `VED.xml`, `RCU.xml` and the named VED/RCU sample CSV files were not found in either checked-out repository. Treat them as required implementation artifacts, regardless of wording elsewhere that suggests installation. Shipped BWS/CCSDLVVWD/CRC/NationalGrid/NVE/SWG schemas are grammar references. [E8]

The provider CSV contract must carry enough detail to generate the required charge lines. The short illustrative VED column list in the recipe is not by itself sufficient for TOU, demand, reactive charges, tax, meter replacement and cross-account credits. Settle the full column/line schema before building the custom XML definitions.

Map provider/company tokens, source account numbers, valid historical Northlake account aliases, meter tokens, commodity/charge descriptions and fiscal periods through normal BIP configuration. Exclude the seller's `SYN-VED-A-0019014`: Northlake has no bills to import for that account. Preserve source identity per invoice and period. For VED export credits, the current Northlake design carries dollars only rather than adding exported kWh to consumption. Units belong to the mapped commodity; a numeric MWh value mapped as kWh can be wrong by 1,000 without an automatic warning. [N2]

Keep the ingestion paths distinct

Entry path	What it does now	Acceptance needed
Provider CSV through WPF Bill Importer	XML schema conversion to flat BIF	Exact headers/charge lines, zero-line handling and generated BIF values. Then separately stage and import.
Native BIF/CSV/XML upload	Native bill upload stages recognized structured files; CSV follows the positional BIF layout	Inspect staged BIP rows, warnings, account/commodity maps and actual imported BillIDs. Provider-specific CSV needs conversion first.
Browser PDF upload	BillImportUploadService sends PDF documents through ScriptString	External extraction/submission is distinct from staged bills. Track return/correlation and failures; no container emulator is implemented here.
UploadJsonSave	Accepts normalized extracted bill JSON and stages BIP data	Author the exact current request model and verify resulting staged rows, mappings, validation and later import. It is not itself PDF extraction or completed bill import.
My Imports workbook	Configured sheet/template/procedure path; some definitions call MyImportToBIP_Save	Select a Northlake definition, generate exact workbook sheets/columns and confirm its staging behavior.
Bill image upload	Attaches document content to an authorized bill or staged-bill/source-document path	Confirm the intended BIP row/BillID relationship, download and restore; do not infer association from matching filenames.

Upload acknowledgement does not prove extraction, import, validation, approval or image association. Test native imports and external submissions separately. [E9-E11]

The file-based `BillImporter` workflow component additionally supports EDI 810 conversion and ZIP input, with chunking, source retention and FileWorkspace recovery. Its `ImportedBillCount` represents accepted staging chunks, not necessarily final Bill rows. Preserve uncertain SQL outcomes for review rather than blindly replaying them. Add EDI/ZIP fixtures after the first BIF path, keeping browser upload capabilities separate from the task component. [E10]

Concrete upload and image contracts

The structured upload route is `POST api/v1/Upload/UploadJsonData` with `{ data, options }`. `options.procName` selects an allowed configured alias, not arbitrary SQL. The `UploadJsonSave` bill envelope requires Customer or CustomerName and a nonempty Documents array. Each document supplies DocId, FileLocation and Extraction; provider information comes from AccountInfo or the extraction root. The default consumer understands account charge collections and MeterLineItems with nested LineItems, including description, quantity, unit/rate and amount aliases. First try that existing normalized shape; a custom SQL handler per fictional provider is not automatically required. [E15]

Freeze an exact sample request and expected staged XML/rows before implementing a producer. Include stable document IDs, customer/provider/account/meter tokens, invoice dates/period, itemized charges and totals. Collective-account fan-out and customer/account concatenation need explicit expected mappings. The current JSON path captures source JSON and document maps after its SQL commit; a later evidence-storage warning can leave the committed upload successful. Check retained evidence separately.

`POST api/v1/Upload/UploadImageData` takes multipart ImageFile plus ImageType and JSON Data. There are three distinct uses:

ImageType	Data / behavior	Association gate
BillImage	companyName, billingAccount, billDate, fileName; stores bytes and returns RelativePath	Put the returned location into the structured bill payload. This call alone does not associate a final BillID.
BillImporterSource	BipFileID, filename and line/source maps	Retains BPF importer evidence. This is separate from the bill's display image.
BillEntry	Actual BillID, FileName and optional label	Attaches to an existing bill. API mode requires a non-impersonated Task principal and Bill Entry Add/Delete permission.

Staging uses FileLocation to create BPFD image metadata; final import copies links to BL bills without duplicating existing links. Those links do not manufacture missing PDF bytes. For PDFs generated after bills already exist, reuse the `BillImage` task: it supports FILENAME or INDEX pairing, lookup configuration and either shared-folder storage or API BillEntry upload. Package an index/lookup contract using real resolved BillIDs; test same-content repeat and changed-content preservation. [E11, E15]

ScriptString submission is yet another contract: HTTPS multipart POST to `/extraction`, with user, company, vendor and files, currently accepting HTTP 200 with count equal to 1. The server supplies routing values. That acknowledgement supplies no extraction ID and has no automatic retry/idempotency guarantee. A compatible stand-in still needs an explicitly designed return-delivery/correlation path into the structured upload API. Do not claim it already completes the round trip. [E16]

PdfPig and Tesseract are registered in-process. The inspected bill-image comparison endpoint compares fields against already-staged bill images and excludes source-document artifacts. This does not establish a general extraction container or PDF-to-bill workflow. `CreateDocument` writes XLSX, not PDFs. [E12]

For a future document workflow, explicitly choose either the current external ScriptString submission/return path or a new, separately scoped parser adapter. The adapter contract must identify source document, extraction state, normalized bill fields, field confidence/provenance where supported, review failures and source-to-staged-row association. Build a deterministic fake response only as a labeled transport test; reserve extraction acceptance for the real chosen parser.

BIP automation and evidence

`BIP_AUTOMATED_IMPORT` exists as a built-in task definition. In Northlake it has no scheduled task item. It does not create provider schemas, account mappings or extraction. Before enabling automation, prove one valid bill, unmapped account/commodity, invalid fiscal period, malformed data, duplicate delivery, estimated/corrected read, meter replacement and the VED cross-account credit case through the normal staging/import path. [E10, N2]

Stage 5 plans 24 document attachments in total: 23 belong to its bill population, with the Ridgeline generated-statement image waiting for Stage 7. The acceptance record must tie each PDF to invoice identity and actual BillID. Bill-image/source-document content and metadata are separate from a SQL backup: `ImageFile` stores a relative filename and content metadata; `Images_tbl` stores associations. Preserve the instance `Content/SavedImages` files and their relative hierarchy when moving the database. Test staged bill-image BPFD to actual bill BL association, with BPF source evidence retained separately. Avoid creating a second bill merely to attach an image. [N2, E11]

Other workflow, import and website work

Tasks and files across stages

Stage 9 already proposes **Bill Download, Import, Validate, Notify Finance**, **Gateway Read, Publish, Weather Regression, Alert**, and **Nightly Campus Load Summary**. Convert these narratives into actual component contracts in small slices. File download is not automatically a gateway collector, gateway publication may require its workflow publish node, and asynchronous intake completion must precede any downstream calculation that assumes the data is stored. Do not rely on hard-coded workflow IDs from an older recipe. [N3, E13]

Include the following producer/consumer pairs in the backlog:

- My Imports XLSX definitions and files: interval data, meter readings, selected hierarchy changes and BIP staging. Northlake's installed Stage 2 sheets do not install every template named in Stage 9.
- My Export, ODBC File Data, Upload Data Files and Upload SQL Results: deterministic expected result sets, formats and a receiving HTTP/SFTP endpoint where required. File creation, transfer receipt and downstream acceptance are separate checks.
- AP/GL output, report delivery, notifications and SQL emailer: expected CSV/PDF/XLSX/HTML attachments, recipient groups, delivery sink evidence and failure behavior. Reuse Mailpit; do not create a mail simulator for each task.
- Document creation/extraction workflows: generation output paths, parser inputs/results, normalization to BIP, review, import, source images and errors. XLSX creation and PDF extraction require different components today.
- Weather normalization, HDD/CDD, warehouse, aggregation, alarms and bill generation: data prerequisites and ordering owned by EEM, without a separate container per calculation.

The Stage 9 task recipe refers to older built-in counts and Daily Administration (17), which is absent from this live task-definition inventory. Refresh those bindings when implementing the slice; do not recreate retired tasks from the document or assume its historical IDs remain authoritative.

Stage 6 lists seven AP/GL export formats and documents missing export registration/builders in its prior Northlake snapshot, including two formats previously assumed to be universal built-ins. Treat those as application/customization prerequisites, not a missing container. Refresh ExportSetup and permission bindings before implementing the receiver. Stage 9's report-delivery plan also needs controlled fanout to its 120 distribution users and 12 groups; test actual attachment content and delivery counts rather than counting configured recipients. [N3]

Provider sites and the Northlake site

Provider manifests describe fictional utilities and website links, but no implemented provider portals were found. Build pages for VED, RCU, Sierra Gas, Helios, Thermal Plant and Ridgeline with the appropriate distinction between supplier invoices, tariffs and EEM-generated statements. Give links stable routes to the same generated artifacts the importers consume. Resolve fictional hostname/URL routing for the target server; a clickable stored URL does not create a website. [N4]

The existing identity customer-site exercises sign-in and session behavior. It is not yet the university portal/API/Power BI demo. Reuse its brand assets and decide whether it should be extended or remain a focused identity harness. The university demonstration should include a campus energy view, a provider/document index, an API-backed view and links to the selected reports. Implement its layout later using existing product/customer patterns; this ADR does not authorize UI changes.

Keep three technical paths separate:

1. **The Web gateway provider emulator** serves the legacy collector's JSON contract.
2. **A Northlake application calling EEM** uses the actual authorized API paths from a server-side client where credentials are needed. Verify company/report access, expired/denied requests, base paths and CORS requirements for any browser call.
3. **Power BI** can consume EEM export tables, while embedding an actual Power BI report is a separate Microsoft service integration.

Current Power BI export routes are `api/v1/powerbi/mfr/{mfrId}/tables` and `/tables/{tableKey}/rows`, with public counterparts under `public/dashboards/{dashboardId}`. Private export permits JWT or PowerBiBasic; that Basic handler uses **username plus session ID**, not a normal password. Public export still requires an eligible public dashboard and public-user/report access. Select report/table keys and permissions from the restored database. [E14]

The API client can exercise `POST api/v1/Authorization` for the established session-ID/API-key exchange, `POST api/v1/IntervalData/GetAnalogDataUtc` for deterministic interval reads, and `POST api/v1/EventBus/Publish` for permitted external event types. Use the deployed base path and configured caller identity. Event publication is not proof that the associated business action completed. [E14]

Microsoft's Web connector supports Basic and JSON GET; its Web API authentication option is not supported in the Power BI service, and Web.Contents POST requests are restricted to anonymous use. Therefore prove the actual Desktop/service refresh recipe rather than assuming a login-POST script works everywhere. Actual report embedding has separate capacity/licensing requirements. A static embedded-looking chart is not Power BI acceptance. Microsoft Web connector, Microsoft embedded capacity

Portable demo deployment and reset

The desired experience is: install a pinned bundle, restore the matching Northlake database and companion content, apply a small environment mapping, verify readiness, then enable accepted tasks. That is achievable, but **database restore plus image start alone is insufficient today**.

Release part	Must travel or be re-established
EEMSuite application/version	Normal installed Web/API/Workers and required task assemblies compatible with the database and workflow versions.
Database checkpoint	Configured-empty acceptance checkpoint and later populated-demo backup, clearly distinguished. Resolve machine/task targets after restore.
Source images and manifests	Pinned image digests, scenario version, seeds, schemas, templates, fonts/brand assets and expected-results manifest.
EEM instance content	SavedImages, imported evidence and any required task-file state; preserve relative names. Do not imply all of this is in SQL.
Per-server configuration	Profiles, worker paths, protocol addresses, runtime gateway enablement, broker, SMTP and API/identity endpoint bindings.
External service state	Identity realm/database/keys, persistent source cursors and selected broker queues; Mailpit history only if it is part of the evidence package.
Environment-held credentials	Supplied through the established deployment mechanism; not baked into images, fixtures, PDFs or database research extracts.
Acceptance evidence	Bundle/scenario/database version, input hashes, run IDs, observed counts/totals, image downloads and known failures.

Stable externally reachable hostnames matter. `localhost` in a user's browser, inside a container and on the Windows worker are different machines/contexts. The older Stage 11 localhost issuer proposal is a local-lab convention, not a general remote demo-server design. Use the identity provider's real public issuer and registered callbacks for the deployment; bind source API/provider URLs and certificate trust consistently.

Use a default small bundle plus opt-in protocol/document/broker capabilities. Starting the source services must not automatically enable every EEM task. A preflight should identify missing profiles, paths, schemas, mappings, API access, host ports and image files without mutating the customer database. Container health is necessary but does not prove any gateway or bill path.

Reset must identify its boundary: replay one input; reset the producer cursor; restore a configured-empty database plus matching workspace/queue state; or restore a populated demo. Do not clear unrelated paths or queues. For asynchronous intake, stop the appropriate consumers, reconcile/quiesce messages and restore matching source state before resuming. Preserve failure artifacts needed for diagnosis. Implement this through a documented runbook first; a generalized reset orchestrator is not needed for the first slice.

Incremental work packages and acceptance gates

These are proposed backlog packages, not implementation commitments or completed work. Each should produce a small runnable result and its evidence before the next scope expands.

Package	Concrete deliverable	Completion gate
P0 - bindings and package contract	Scenario/version manifest, path/endpoint map, source IDs, fixed/live clock rules, backup/content inventory	Resolve against Northlake; no unexplained installed IDs or hidden host paths.
P1 - first source image	Existing AcquiSuite and bill-PDF generators packaged; health and fixture manifest; one delivery root	Complete/replay/gap/backfill accepted through the normal worker; PDF hashes/totals reproducible.
P2 - file families	Seven FIG serializers, Spinwave, MDEF and MV9 binary encoders, small cases before full history	Parser-specific mapping/scaling/time/status behavior and stored results match expected manifests.
P3 - first bill pipeline	VED/RCU schemas, detailed provider CSV, BIF/JSON contracts, mappings and PDF associations	One invoice through conversion/staging/review/import/image download, then named negative and duplicate cases.
P4 - handheld	Read-type/route prerequisites, exported route, synthetic completion files and event sidecars	Reading/usage/rollover/skip results and persisted HMR events; state-aware replay.
P5 - live protocols and historian	Modbus, BACnet, ODBC source and Web provider endpoints	Windows worker can connect; accepted values, timeout/recovery and live-versus-history behavior.
P6 - document and operational workflows	Selected parser path, normalized JSON staging, download/validate/notify; My Imports/Exports and delivery	Trace one source document/file through each step, failures, review and final output; no success-by-submission claims.
P7 - provider and university sites	Working provider links/documents and Northlake API/report views	Links resolve on the demo host; scoped access and actual API/Power BI refresh or embed demonstrated.
P8 - portable release rehearsal	Pinned Compose bundle, populated database/content package and restore runbook	Clean target-host restore with limited environment configuration, selected schedules and end-to-end evidence.

P3 can proceed beside P2 after the common data/manifest contract. P4 waits on HMR product configuration. P5 can be delivered one protocol at a time. P6 and P7 should follow the chosen operational and third-party acceptance cases, rather than holding up Stage 3's first successful feed.

For each slice, future validation must use the repository's testing policy and the narrow relevant parser/service/contract tests, then the normal installed acceptance path. Local unit tests, synthetic HTTP responses, SQL row checks and rendered browser/desktop acceptance are distinct evidence levels. This documentation pass ran none of those acceptance suites.

Open decisions and implementation blockers

- Design the new document-processing service's input/output API and chosen extraction engine. The user confirmed no document container was built. Reuse the PDF generator and relevant extraction code, and prove the normalized JSON return and evidence-retention path before adding a new renderer.
- Choose the first deployment host's container runtime/networking capability, source hostname convention and host delivery root versus network fetch. Prove BACnet UDP and Modbus TCP from the actual worker.
- Resolve HMR read-type catalog and header setup through supported application/product changes before implementing route acceptance. Do not seed ad hoc direct bindings.
- Define the detailed VED/RCU file schema and normalized UploadJsonSave payload, including source-document correlation; the current examples are not a complete invoice contract.
- Select the synthetic historian's source schema/profile on existing infrastructure. Do not add SQL container scope or write into destination archives as a source shortcut.
- Decide whether offline weather is required. If yes, plan an explicit production-supported replay seam; the current NOAA/ECCC client cannot just target a local host.
- Choose initial public/private MFR export cases and whether actual Power BI hosting/embedding is required. Treat credentials, tenant/capacity and remote refresh as deployment prerequisites.
- Reconcile the existing recipes' stale descriptions during their implementation slices: binary MV9, Web gateway coverage, retired task IDs/counts, unpublished provider schemas, missing Northlake MeterReadings import, and interval-start/end distinctions.
- Define one packaged demo checkpoint with matching images/workspaces/source state. A two-year history generator must satisfy bills, rates, periods, gaps, meter swaps, outages and Ridgeline credit reconciliation, not merely fill every interval with random values.

Evidence index

Paths below are relative to roots N and E defined earlier. They identify the inspected contracts so subsequent work can refresh the evidence without relying on this prose. Line numbers are useful entry points at the research snapshot, not durable identities.

Northlake plans and generators

- **N1:** `implementation/stage-3-gateways-and-data/README.md`; `02-gateways.yaml`; `03-workflows-and-tasks.yaml`; `04-acceptance-runs.yaml` (notably MV9 at line 189); `05-handheld-routes.yaml`; `01-checkpoint.yaml`. AcquiSuite generation: `generators/northlake_packet.py`, around lines 669-717.
- **N2:** `implementation/stage-5-bills/02-bill-importer-sources.yaml`, `03-bills.yaml`, `04-bill-images-and-notes.yaml`; `generators/render_bills.py`; `data/bills/northlake-itemized-bills-v1.yaml`; `data/providers/`; Stage 5 and Stage 7 chapter sources under `generators/documents/sources/implementation/`.
- **N3:** `implementation/stage-9-operations/04-tasks-and-workflows.yaml`, `06-report-delivery.yaml`, `07-handheld-and-importers.yaml`; `implementation/stage-6-financial/04-ap-export-formats.yaml`; Stage 9 chapter source. Live task/My Import inventories in this record supersede older count/ID assumptions for this research snapshot.
- **N4:** `identity/README.md`, `identity/compose.yml`, identity configuration/customer-site sources and `identity/branding/README.md`; `data/providers/*.yaml`; `generators/documents/sources/implementation/stage-11-third-party/stage-11-third-party.md`.

EnergyHippo production contracts

- **E1:** `src/Tasks/EEMSuite.Tasks.Gateways/GatewayServiceCollectionExtensions.cs:21`; `src/Domain/EEMSuite.Domain/Admin/Gateways/Types/KnownGatewayTypes.cs` and `GatewayObjectKind.cs`.
- **E2:** `src/Tasks/EEMSuite.Tasks.Gateways/Fig/FIGGatewayComponent.cs:30`, `FigFormatParsers.cs`; `AcquiSuite/AcquiSuiteGatewayComponent.cs:277`; `Spinwave/SpinwaveGatewayComponent.cs:223`, under the same gateway project. Synthetic examples: `tests/Tasks/EEMSuite.Tests.Tasks.Gateways/TestData/Fig/`.
- **E3:** `src/Tasks/EEMSuite.Tasks.Gateways/Mv90/MV90GatewayComponent.cs:126`, especially lookup at line 188; `Mv90BinaryRecordReader.cs`, `MdefFileParser.cs`, `Mv9FileParser.cs`; `tests/Tasks/EEMSuite.Tests.Tasks.Gateways/Mv90BinaryFixtureBuilder.cs`.
- **E4:** `src/Tasks/EEMSuite.Tasks.Gateways/Neptune/NeptuneGatewayComponent.cs:70`, `NeptuneUploadFileParser.cs`; `Mvrs/MVRSGatewayComponent.cs:79`, `MvrsUploadFileParser.cs`; corresponding parser tests; `HmrEvents/HmrEventsPublishComponent.cs:49`.
- **E5:** `src/Tasks/EEMSuite.Tasks.Gateways/Shared/Helpers/GatewayWorkspace.cs:23`, source claim/finalization and state-sensitive abandoned-run policy; `src/Domain/EEMSuite.Domain/Tasks/FileWorkspace/Models/TaskFileWorkspaceOptions.cs`; `src/Domain/System/EEMSuite.Domain.Files/Workspace/FileWorkspace.cs`.
- **E6:** Gateway project `Modbus/ModbusGatewayComponent.cs:79`, `Bacnet/BACnetGatewayComponent.cs:54`, `Odbc/GatewaySourceQueryExecutor.cs:27`, `Web/WebGatewayClient.cs:95`; `src/Domain/EEMSuite.Domain/API/RestDataContract.cs`.
- **E7:** Gateway project `Noaa/README.md`, `Noaa/NoaaHttpClient.cs:35`, Aeris and Fake implementations; `src/Domain/EEMSuite.Domain/Configuration/Settings/GatewayRuntimeOptions.cs`.
- **E8:** `src/Apps/EEMSuite.Apps.BillImporter/Schemas/CRC.xml`; `BillImporterMainWindowViewModel` provider loading and `BillImporter:Utilities` configuration in that project.
- **E9:** `src/Domain/Services/EEMSuite.Domain.Services/Billing/Bills/BillImportUploadService.cs`; `src/Web/EEMSuite.Web.Endpoints/Controllers/BillImporterController.cs`; current upload request/result models and native import implementation.
- **E10:** `Database/EEMSuite/Procs/BillImportPro_FileData_Validate.sql`, `BillImportPro_FileData_Import.sql`, `BillImportPro_ImportTask.sql`, `MyImportToBIP_Save.sql`; built-in task catalog entries for `BIP_AUTOMATED_IMPORT`; `src/Tasks/EEMSuite.Tasks.Components/BuiltInWorkflows/bill-importer.json`; `src/Tasks/EEMSuite.Tasks.BillImporter/Components/BillImporterComponent.cs`.
- **E11:** `src/Domain/Services/EEMSuite.Domain.Services/Billing/Bills/BillImporterSourceDocumentService.cs`; `src/Domain/EEMSuite.Domain/Billing/Bills/Interfaces/IBillImporterSourceDocumentRepository.cs`; image upload/repository and instance-content path contracts. Live `ImageFile` and `Images_tbl` metadata were inspected without reading document content.
- **E12:** `src/Domain/TextExtraction/Implementations/PdfPigTextExtractionProvider.cs`; `src/Domain/TextExtraction.Tesseract/`; web registration and bill-image comparison controller; `src/Tasks/EEMSuite.Tasks.Documents/Components/CreateDocumentComponent.cs` and `Services/XlsxDocumentWriter.cs`.
- **E13:** `src/Tasks/EEMSuite.Tasks.Components/BuiltInWorkflows/` including `my-import.json`, `my-export.json`, `upload-data-files.json`, `upload-sql-results.json`, `odbcfiledata-gateway.json`, `maintenance-sql.json`; `BuildScripts/Local/Docker/local-services-docker-compose.yml` and `README.md`.

- **E14:** `src/Web/API/Controllers/PowerBiController.cs:18`, `AuthorizationController.cs:142`; `src/Web/API/Services/PowerBi/PowerBiExportService.cs:70`; `src/Web/API/Models/Security/PowerBiBasicAuthenticationHandler.cs:43`; `src/Web/API/Program.cs:342`; shared `IntervalDataController.cs` and `EventBusController.cs` under `src/Web/EEMSuite.Web.Endpoints/Controllers/`; `src/Domain/Services/EEMSuite.Domain.Services/Reporting/Reporting/PowerBiExportAccessService.cs`.
- **E15:** `src/Web/API/Controllers/UploadController.cs:61`; `src/Web/API/Models/Upload/UploadService.cs:375` and the adjacent request models; `Database/EEMSuite/Procs/UploadJsonSave.sql`, `UploadJsonConsume_Default.sql`, `UploadJsonBuildBIP.sql`, `BillImportPro_FileData_Save.sql`; `src/Tasks/EEMSuite.Tasks.BillImage/Models/BillImageSettings.cs` and `Orchestration/BillImageStore.cs`.
- **E16:** `src/Integrations/EEMSuite.Integrations.ScriptString/ScriptStringClient.cs:15`; upload receipt semantics in `BillImportUploadModels.cs` and `BillImportUploadService.cs`.

External references are linked beside their associated claims. Recheck their supported versions and deployment requirements when implementing a selected slice. The document is a build inventory and proposed direction; installed and remote acceptance remains outstanding.